Shield Design System — Rebuilding the Foundation, Token by Token

Role: UI Designer, Design Systems | **Timeline:** 2023–2025 | **Scope:** 6 products, 11 teams, enterprise cybersecurity

---

Overview

The Shield Design System was a shared Figma file that nobody trusted. Designers detached components at a 40% rate. Products looked different from each other. Dark mode was handled per-product with hardcoded values. Federal accessibility audits produced inconsistent results, putting government contracts at risk.

I rebuilt the system from the ground up. Audited every component, built a structured token architecture, established governance, migrated documentation, and drove adoption from 2 products to 6 across 11 teams.

Detach rate: 40% → under 1%

Component usage: 7,000 → 61,000+ insertions per quarter

Token coverage: 791 variables across light and dark modes, 94,000+ tokens in production

Adoption: ~90% across teams

Compliance: AA+ contrast, Section 508 alignment

What I Walked Into

The design system existed, technically. But it was in maintenance mode. Updates were reactive and scattered. Design and engineering operated in silos. There was

no token layer, no governance, and no shared understanding of what a component was supposed to do versus what it accidentally did.

I audited the v3.0 library and found 100+ disconnected components with deep misalignment between what Figma showed and what engineering had built. Each team styled things their own way. Developers hardcoded color values. Designers relied on manual overrides. Handoffs were fragile. The same UI got built twice because nobody trusted the source.

This wasn't a cosmetic problem. The company sells to US federal agencies under contracts that require Section 508 compliance and VPAT maintenance. Inconsistent UI and stale accessibility documentation create real disqualification risk during procurement reviews.

The Audit

I started by classifying every SDS color, radius scale, and typographic style as decorative, functional, or state-driven. That behavioral view of the system was the thing that revealed where values were duplicated, misaligned, or carrying hidden intent nobody had named.

The audit also surfaced naming conflicts. The clearest example was "Tiles vs Cards." Teams used both terms interchangeably, which blurred accessibility focus state requirements. I resolved it by defining Tiles as navigation triggers and Cards as content containers. That one decision clarified component scope and accessibility behavior across five products.

Another pattern: global alert banners were being misused for inline feedback. Teams had no alternative, so every warning, every validation message, every contextual note got pushed into a full-width banner at the top of the screen. Users stopped reading them. I introduced an Inline Alert component, and global banner misuse dropped by 70%.

Building the Token Architecture

The audit made it clear that fixing components without fixing the underlying value system would just recreate the same drift. Six products running on different

frontend frameworks needed an abstraction layer between raw values and component-level usage. So I built a structured token architecture from scratch.

Three Layers

1. **Primitives.** Foundational values with no context. `color.navy.700` → `#1C3D5A`. `spacing.sm` → `8px`. These define palettes and scales.
2. **Semantics.** Translates primitives into intent. `color.text.primary` → `color.navy.700`. This layer defines purpose without dictating how a specific component renders.
3. **Component Tokens.** Applied styles for components. `button.primary.default-background.color`. These reference semantics (or primitives in the current hybrid model) to ensure predictability.

I implemented the full token set in Token Studio, synced with Figma, and exported as JSON or SCSS for Angular. Updates propagate in real time. Theme generation for light and dark modes is automated.

Naming That Explains Itself

I introduced a naming pattern designed to make every token self-explanatory without needing documentation:

- **Semantic tokens:** `{category}.{role}.{state}.{mode}`
- **Component tokens:** `{component}.{variant}.{state}-{element}.{attribute}`

This means designers and developers can read a token name and understand what it controls, where it applies, and how it behaves across modes. It also means automation tools can parse the taxonomy without special logic.

Interactive States

All core components follow consistent states: `default`, `hover`, `focus`, `disabled`, `error`, and `active`. These are shared between Figma and code. One set of definitions, one-to-one mapping. No ambiguity about what "hover" means for a button versus a card versus a nav item.

The Typography Problem

During the build, the typography pipeline exposed a real limitation in the Figma-to-Token Studio workflow. Figma had difficulty interpreting text variables consistently, especially converting style names like Bold or Medium into numeric weight values (700, 500). This created mismatches between design intent and what Token Studio recognized as valid input. I restructured the typography token names and aligned the weight logic directly in code. Not elegant, but it eliminated typography drift between tools.

Architectural Tradeoffs

To keep engineering velocity during integration, I adopted a hybrid structure: component tokens map directly to primitives in some cases, skipping the semantic layer. This ensured immediate stability.

But I documented and championed a roadmap toward the strict three-layer model. The current state works. The future state scales better. Moving component tokens to reference semantics will fully decouple intent from implementation, so updates to shared meanings automatically propagate everywhere.

- **Current:** Component tokens → Primitives (speed and stability)
- **Future:** Component tokens → Semantics → Primitives (scalability and meaning)

This was a conscious decision, not a shortcut. I needed teams to trust the system before I could ask them to trust additional abstraction.

Constraints I Worked Around

Legacy frameworks. Two products ran older UI kits (PrimeNG, custom Angular) that could not consume tokens natively. I created migration guides mapping hardcoded values to semantic tokens, keeping adoption low-effort.

No dedicated frontend engineer. I scoped component specs to minimize ambiguity. Figma component properties mapped 1:1 to code props so there was less room for interpretation gaps.

Federal compliance pressure. The company maintains 16 VPATs for government procurement. Accessibility failures can disqualify the company from contracts.

This wasn't theoretical. It shaped every decision about contrast ratios, focus states, and state behavior.

Governance and Documentation

Governance matured into a structured system rather than ad-hoc updates. Version-controlled releases and README change logs formalized how updates ship and how history is preserved.

I modernized the documentation layer and migrated guidance into a unified Storybook + README framework. Usage guidance, token references, and code demos now live together as a single, versioned source of truth. Documentation became live, versioned, and integrated. Designers and engineers read the same source.

I also introduced a Proposal + Review workflow that formalized how components enter the system. No more duplicated components under different names. No more "I just made my own version."

Adoption and Evidence

I measured adoption using Figma Analytics and a Token Usage Tracker, monitoring insertions, detaches, and usage patterns per team.

- **90% adoption** across teams
- **Component usage:** 7,000 → 61,000+ in 90 days
- **Token usage:** Exceeded 94,000
- **Detach rates:** 40% → under 1%

Adoption expanded from two products to six across eleven teams. The visual modernization (AA+ contrast, dual-theme tokens for light and dark modes) was the unlock. Teams could finally see the system working for them instead of against them.

Product Impact

Product 1 (Linux) leads with a 60% conformance rate, validating the scalability of the token architecture.

- Unified look and feel across Linux management screens
- Resolved contrast gaps and achieved WCAG compliance
- Global updates now occur through tokens, reducing regression risk
- Shared tokens shortened design-to-dev handoffs and improved release cadence

Redesigned experiences consistently scored high on usability and customer satisfaction. Task success increased across complex workflows. User errors decreased. Onboarding improved as screens followed consistent patterns.

AI-Assisted Validation

I ran an experiment using AI to stress-test the token system's structure and naming logic. I prompted models to interpret and generate tokens from the established patterns. The results confirmed the taxonomy was predictable and machine-readable. AI could infer intent and hierarchy without documentation.

More useful: the experiment surfaced minor inconsistencies I refined before rollout. It proved the system is not only human-friendly but ready for future automation like theme generation and design linting.

What Actually Changed

Before:

- Maintenance mode. Updates were reactive and scattered.
- Disconnected. Design and engineering operated in silos.
- Ungoverned. Shield was a shared resource, not a product.

After:

- Governed product with clear lifecycle stages and a governance model.
- Unified source of truth. Tokens, components, and docs are version-controlled together.
- Self-sustaining. Governance is shared, with direct connections between Figma, Storybook, and code.

Business Impact

Revenue protection. Stale accessibility documentation and inconsistent UI create disqualification risk during federal procurement reviews. A governed, accessible design system is procurement infrastructure.

Engineering time recovered. Every detach meant engineering building custom UI instead of using the system. At scale across 11 teams, that was hundreds of hours per quarter spent rebuilding what should already exist. Reducing detach to under 1% returned that time to feature work.

Faster delivery. Surfaces that previously required weeks of custom component work now ship using system components. Design reviews shortened because the system enforces consistency before the review happens.

Reduced onboarding cost. New designers inherit a governed system with versioned docs instead of tribal knowledge.

What I Still Question

Adoption metrics confirm teams use the system. They do not confirm teams understand it well enough to extend it correctly without me. The gap between usage and genuine shared ownership is still open.

The hybrid token structure (component → primitive, skipping semantic in places) works today. Whether it holds as the system grows is the next real test.

What's Next

- Expand token coverage to motion and data visualization components
- Implement automated audit tools for accessibility and token usage
- Roll out onboarding workshops for new teams joining Shield's ecosystem
- Migrate remaining component tokens to reference semantics (completing the three-layer model)